

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Зотеев Максим

БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать REST API сервиса аренды недвижимости на основе результатов ДЗ1 (ERD) и ДЗ2 (OpenAPI-спецификация). Использовать подходящий boilerplate, реализовать модели, представления и контроллеры, продемонстрировать полностью работающее приложение.

Ход работы

1. Технологический стек

В качестве основы выбран стек Express.js + TypeORM + TypeScript. Express обеспечивает минимальный и расширяемый слой HTTP-маршрутизации, TypeORM - зрелый ORM с поддержкой декораторов и автоматической генерации схемы по метаданным сущностей, TypeScript - статическую типизацию и безопасный рефакторинг. В качестве СУБД используется PostgreSQL 16, запускаемая рядом с API в Docker Compose. Для аутентификации применяются JWT (jsonwebtoken) и bcryptjs для хеширования паролей.

2. Структура проекта

Проект организован по слоям: entities - TypeORM-модели, controllers - бизнес-логика, routes - HTTP-маршрутизация, middleware - сквозные обработчики (JWT-аутентификация, обработка ошибок), utils - вспомогательные модули (хеширование паролей, работа с JWT, пагинация, DTO-мапперы). Точка входа src/index.ts инициализирует подключение к БД, запускает seed-скрипт и поднимает Express-приложение.

3. Модели данных (TypeORM entities)

Сущности соответствуют ERD из ДЗ1: User, PropertyType, Location, Property, PropertyPhoto, Amenity, Rental, Message. Связи реализованы декораторами @ManyToOne, @OneToMany, @OneToOne, @ManyToMany. Связь многие-ко-многим между Property и Amenity описана @JoinTable с именем property_amenities, что соответствует связующей таблице из ERD. Свойство eager: true используется там, где связанные данные всегда нужны при чтении (тип недвижимости, локация, удобства, владелец сделки).

4. Аутентификация и авторизация

Регистрация и вход возвращают JWT access_token со сроком жизни, задаваемым через переменную окружения JWT_EXPIRES_IN. Middleware authRequired проверяет заголовок Authorization: Bearer <token>, декодирует payload (sub - идентификатор пользователя, role - роль) и кладёт его в req.user. Защищённые операции (создание объекта, аренда, сообщения) используют authRequired, а

создание и редактирование объекта дополнительно проверяет, что пользователь - арендодатель (`role = landlord`) и что он является владельцем редактируемой записи.

5. Эндпоинты API

Полный список маршрутов соответствует OpenAPI-спецификации из Д32. В категорию Auth входят `/auth/register` и `/auth/login`. В категорию Users - `/users/me` (GET/PATCH), `/users/me/properties`, `/users/me/rentals`. В категорию Properties - поиск с фильтрами `/properties` (GET), CRUD `/properties/{id}`, загрузка и удаление фото `/properties/{id}/photos`, справочники `/property-types` и `/amenities`. В категорию Rentals - создание заявки POST `/properties/{id}/rentals`, получение сделки, завершение и отмена. В категорию Messages - `/rentals/{id}/messages` (GET/POST).

6. Фильтрация и справочник удобств

Эндпоинт GET `/properties` поддерживает параметры `property_type_id`, `city`, `district`, `price_min`, `price_max`, `rooms_min`, `rooms_max`, `amenity_ids` (повторяющийся параметр), `sort_by` и пагинацию `page/size`. Фильтрация по удобствам реализована через подзапрос к связующей таблице `property_amenities` с группировкой по `property_id` и условием `COUNT(DISTINCT amenity_id) = N`, что гарантирует наличие у объекта ВСЕХ указанных удобств, а не хотя бы одного. Справочник удобств доступен через GET `/amenities` и предзаполняется при старте приложения (Wi-Fi, парковка, кондиционер, стиральная машина, посудомоечная машина, телевизор, лифт, балкон, отопление, можно с животными).

7. Обработка ошибок и валидация

Используется класс `HttpError` и набор фабрик (`badRequest`, `unauthorized`, `forbidden`, `notFound`, `conflict`) для формирования ответов в формате `ErrorResponse` из Д32 (`{error, message}`). Централизованный middleware `errorHandler` перехватывает такие ошибки и преобразует их в HTTP-ответы соответствующего кода. Асинхронные контроллеры оборачиваются хелпером `asyncHandler`, который передаёт отклонённые промисы в `next()`, чтобы их поймал `errorHandler`. Валидация входных данных выполняется непосредственно в контроллерах на этапе приёма тела запроса.

8. Запуск в Docker Compose

Файл `docker-compose.yml` описывает два сервиса: `db` (PostgreSQL 16 Alpine) с `healthcheck` и `api` (собирается из локального Dockerfile, Node 20). Сервис `api` зависит от `db` через `condition: service_healthy`, что гарантирует старт приложения только после готовности базы. Переменные окружения подключения к БД и секрет JWT прокидываются через `environment`. Для первого запуска достаточно

команды `docker compose up --build`; при старте автоматически создаются все таблицы (`synchronize: true`) и заполняется справочник типов и удобств.

Заключение

В рамках лабораторной работы был реализован REST API сервиса аренды недвижимости на стеке Express + TypeORM + TypeScript, полностью соответствующий ERD из Д31 и OpenAPI-спецификации из Д32. Реализованы аутентификация по JWT, разграничение ролей арендодатель/арендатор, CRUD объектов недвижимости с фильтрацией (включая фильтрацию по удобствам), управление сделками аренды, переписка в рамках сделки и справочники. Приложение упаковано в Docker Compose вместе с PostgreSQL и запускается одной командой.